

Assignment 3: Flights of New York

Zeynep Cakez

2024-09-14

Exercise 1

- i. How many rows and columns does this dataset have?

There are 19 columns and 336,776 rows.

- ii. What does a single row in this dataset represent?

A single flight.

- iii. What is the difference between the information contained in the `arr_time` and `sched_arr_time` columns? (Take a look at the column descriptions)

`arr_time` is the actual time that the plane arrived, `sched_arr_time` is when it was supposed to arrive.

- iv. Airplanes are reused across many different flights. Which column(s) would be helpful to use in identifying individual airplanes?

Yes, they were reused. The `tailnum` column helps with this.

Exercise 2

```
flights %>%  
  select(year, month)
```

```
## # A tibble: 336,776 x 2  
##   year month  
##   <int> <int>  
## 1  2013     1  
## 2  2013     1  
## 3  2013     1  
## 4  2013     1  
## 5  2013     1  
## 6  2013     1  
## 7  2013     1  
## 8  2013     1  
## 9  2013     1  
## 10 2013     1  
## # i 336,766 more rows
```

Exercise 3

```
flights %>%  
  select(year: day)
```

```
## # A tibble: 336,776 x 3  
##   year month   day  
##   <int> <int> <int>  
## 1  2013     1     1  
## 2  2013     1     1  
## 3  2013     1     1  
## 4  2013     1     1  
## 5  2013     1     1  
## 6  2013     1     1  
## 7  2013     1     1  
## 8  2013     1     1  
## 9  2013     1     1  
## 10 2013     1     1  
## # i 336,766 more rows
```

The colon selects only the columns from year to day.

Exercise 4

```
flights %>%  
  arrange(air_time, distance) %>%  
  select(air_time, distance)
```

```
## # A tibble: 336,776 x 2  
##   air_time distance  
##   <dbl>    <dbl>  
## 1      20      116  
## 2      20      116  
## 3      21       80  
## 4      21       80  
## 5      21       94  
## 6      21      116  
## 7      21      116  
## 8      21      116  
## 9      21      116  
## 10     21      116  
## # i 336,766 more rows
```

i. Does it look like both the `air_time` and `distance` columns were sorted?

No, they weren't.

ii. Which column was sorted first?

`air_time`

iii. What happens if you reverse the order you specify the columns in `arrange()`?

Then distance gets arranged first.

Exercise 5

```
flights %>%
  arrange(desc(dep_delay)) %>%
  select(dep_delay, carrier, flight)
```

```
## # A tibble: 336,776 x 3
##   dep_delay carrier flight
##   <dbl> <chr>    <int>
## 1     1301 HA         51
## 2     1137 MQ       3535
## 3     1126 MQ       3695
## 4     1014 AA        177
## 5     1005 MQ       3075
## 6       960 DL       2391
## 7       911 DL       2119
## 8       899 DL       2007
## 9       898 DL       2047
## 10      896 AA        172
## # i 336,766 more rows
```

The longest departure delay was carrier HA and flight number 51

Exercise 6

```
flights %>%
  mutate(average_speed = distance / (air_time / 60)) %>%
  select(year: dep_delay, average_speed)
```

```
## # A tibble: 336,776 x 7
##   year month   day dep_time sched_dep_time dep_delay average_speed
##   <int> <int> <int>   <int>         <int>         <dbl>         <dbl>
## 1  2013     1     1     517             515           2           370.
## 2  2013     1     1     533             529           4           374.
## 3  2013     1     1     542             540           2           408.
## 4  2013     1     1     544             545          -1           517.
## 5  2013     1     1     554             600          -6           394.
## 6  2013     1     1     554             558          -4           288.
## 7  2013     1     1     555             600          -5           404.
## 8  2013     1     1     557             600          -3           259.
## 9  2013     1     1     557             600          -3           405.
## 10 2013     1     1     558             600          -2           319.
## # i 336,766 more rows
```

i. Where does the new column you just computed show up in the dataset and what is the name

of this new column?

It is the last column, and its called `average_speed`.

ii. What part of the code is controlling the name of the new column?

Where it equals to the columns that its going to mutate.

Exercise 7

```
flights %>%
  mutate(dep_time_hour = dep_time %/% 100,
         dep_time_minute = dep_time %% 100,
         dep_time_minutes_midnight = (dep_time_hour * 60) + dep_time_minute) %>%
  select(dep_time_hour: dep_time_minutes_midnight)
```

```
## # A tibble: 336,776 x 3
##   dep_time_hour dep_time_minute dep_time_minutes_midnight
##           <dbl>           <dbl>                <dbl>
## 1             5             17                   317
## 2             5             33                   333
## 3             5             42                   342
## 4             5             44                   344
## 5             5             54                   354
## 6             5             54                   354
## 7             5             55                   355
## 8             5             57                   357
## 9             5             57                   357
## 10            5             58                   358
## # i 336,766 more rows
```

Exercise 8

```
flights %>%
  filter(arr_delay < 0, carrier == "AA") %>%
  select(year: arr_time ,arr_delay: carrier)
```

```
## # A tibble: 20,769 x 9
##   year month   day dep_time sched_dep_time dep_delay arr_time arr_delay
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>     <dbl>
## 1  2013     1     1     606           610           -4     858       -12
## 2  2013     1     1     628           630           -2    1137        -3
## 3  2013     1     1     656           659           -3     949       -10
## 4  2013     1     1     659           700           -1    1008        -7
## 5  2013     1     1     712           715           -3    1023       -12
## 6  2013     1     1     739           745           -6     918       -12
## 7  2013     1     1     753           755           -2    1056       -14
## 8  2013     1     1     803           810           -7     903       -22
## 9  2013     1     1     840           845           -5    1311       -39
```

```
## 10 2013      1      1      940      945      -5      1119      -11
## # i 20,759 more rows
## # i 1 more variable: carrier <chr>
```

Exercise 9

```
flights %>%
  group_by(carrier) %>%
  summarize(average_arr_delay = mean(arr_delay, na.rm = TRUE)) %>%
  arrange(desc(average_arr_delay))
```

```
## # A tibble: 16 x 2
##   carrier average_arr_delay
##   <chr>          <dbl>
## 1 F9             21.9
## 2 FL             20.1
## 3 EV             15.8
## 4 YV             15.6
## 5 OO             11.9
## 6 MQ             10.8
## 7 WN              9.65
## 8 B6              9.46
## 9 9E              7.38
## 10 UA             3.56
## 11 US             2.13
## 12 VX             1.76
## 13 DL             1.64
## 14 AA             0.364
## 15 HA            -6.92
## 16 AS            -9.93
```

i. Which airline carrier had the longest arrival delays on average?

F9 has the longest delay, 21.92.

ii. Which airline carrier had the shortest arrival delays on average?

AS has the shortest delay, -9.93.

```
flights %>%
  group_by(carrier) %>%
  summarize(
    average_arr_delay = mean(arr_delay, na.rm = TRUE),
    average_dep_delay = mean(dep_delay, na.rm = TRUE)
  )
```

```
## # A tibble: 16 x 3
##   carrier average_arr_delay average_dep_delay
##   <chr>          <dbl>          <dbl>
## 1 9E             7.38           16.7
## 2 AA             0.364          8.59
```

## 3 AS	-9.93	5.80
## 4 B6	9.46	13.0
## 5 DL	1.64	9.26
## 6 EV	15.8	20.0
## 7 F9	21.9	20.2
## 8 FL	20.1	18.7
## 9 HA	-6.92	4.90
## 10 MQ	10.8	10.6
## 11 OO	11.9	12.6
## 12 UA	3.56	12.1
## 13 US	2.13	3.78
## 14 VX	1.76	12.9
## 15 WN	9.65	17.7
## 16 YV	15.6	19.0

Exercise 10

```
late_flights_to_miami <- flights %>%
  filter(dest == "MIA" & arr_delay > 0) %>%
  select(arr_delay, carrier)
```

Exercise 11

New Data Set

```
monthly_delays <- flights %>%
  group_by(month, carrier) %>%
  summarize(
    arrival_delay = mean(arr_delay, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  spread(carrier, arrival_delay) %>%
  select(-`9E`)
```

```
pivoted_monthly_delays <- monthly_delays %>%
  pivot_longer(cols = -month, names_to = "airline", values_to = "arrival_delay")
```

```
library(ggplot2)
qplot(x = month,
      y = arrival_delay,
      color = airline,
      geom = "line",
      data = pivoted_monthly_delays)
```

